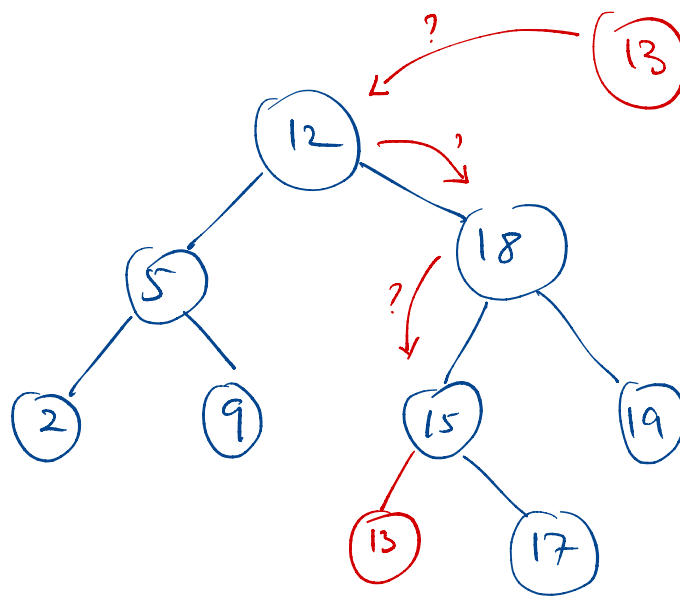


Binary Search Tree - III

Insertion : Causes the dynamic set to be modified.

→ Insertion must ensure that the BST property continues to hold

Example :



Input: a BST node Z whose key has been set and $Z.\text{left} = \text{NIL}$
 $Z.\text{right} = \text{NIL}$



Tree - Insert (T, Z)

$x = T.\text{root}$ // Node being compared with Z
 $y = \text{NIL}$ // y will be the parent of Z
while $x \neq \text{NIL}$

```
    y = x
    if  $Z.\text{key} < x.\text{key}$ 
         $x = x.\text{left}$ 
    else
         $x = x.\text{right}$ 
```

$Z.p = y$ // found the location to insert Z with parent y

if $y == \text{NIL}$ // Tree was empty

$T.\text{root} = Z$

else if $Z.\text{key} < y.\text{key}$

$y.\text{left} = Z$

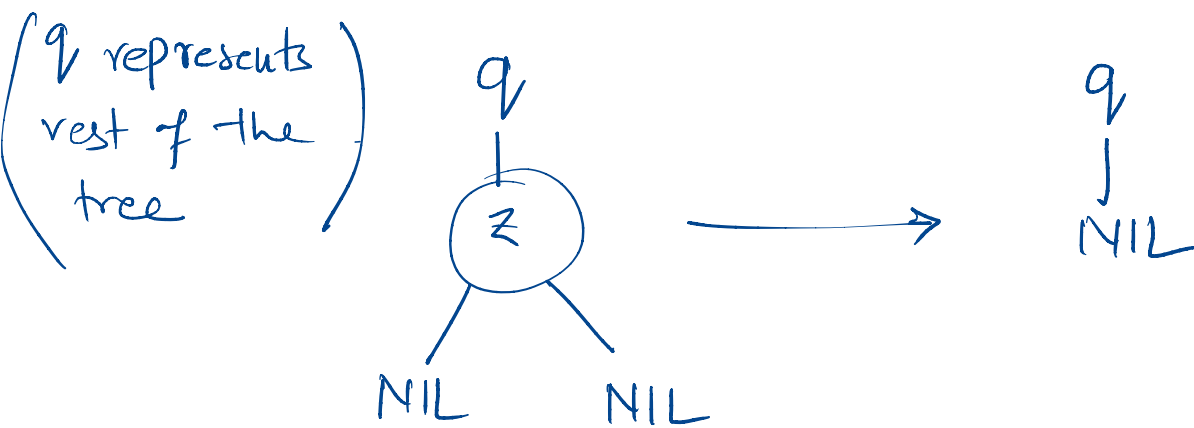
else $y.\text{right} = Z$

$\downarrow$
 $O(h)$
Complexity

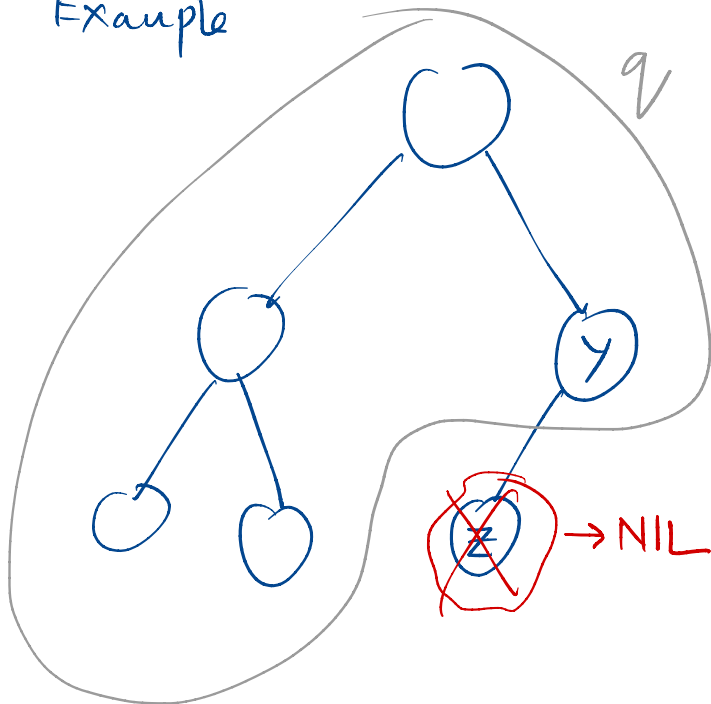
Deletion : Delete a node z from the
BST T

3 basic cases:

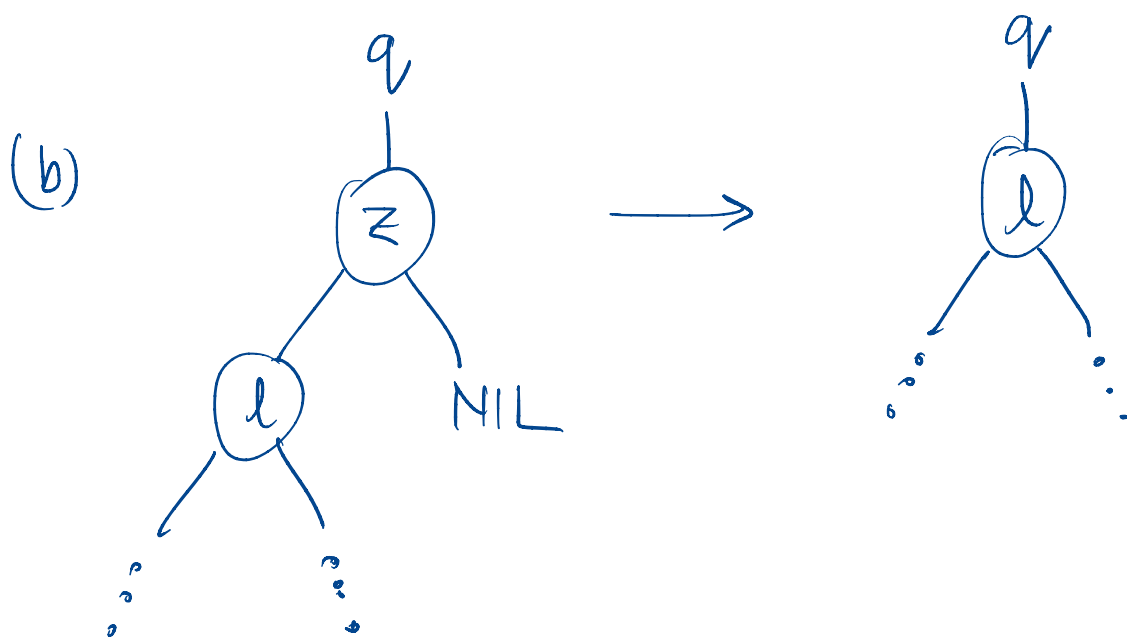
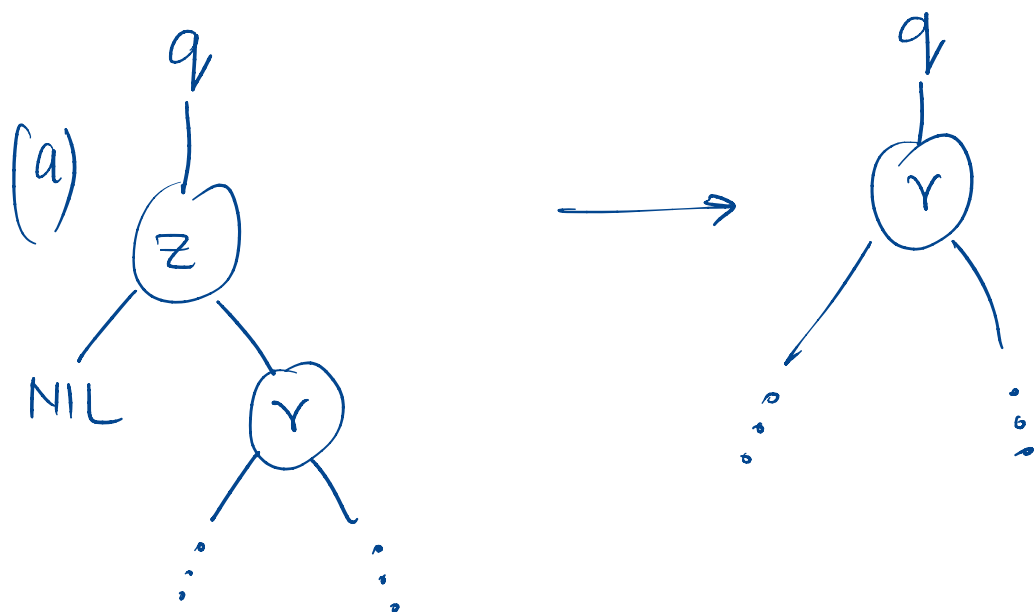
(1) If z has no children then simply
remove it by modifying its parent
to replace z with NIL as child



Example



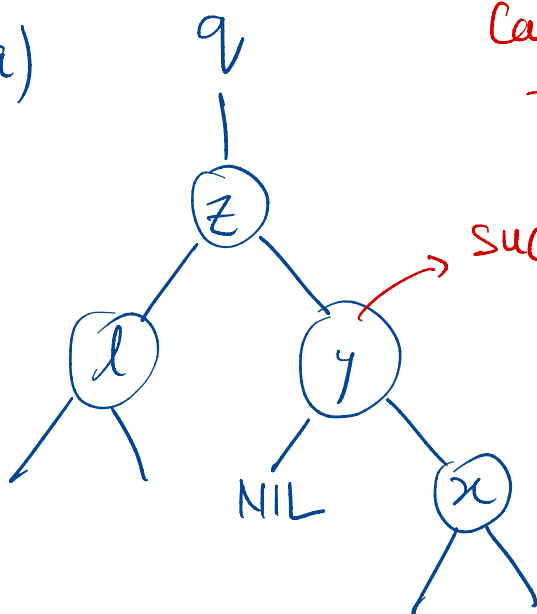
(11) If z has just 1 child, then elevate that child to take z 's position in the tree by modifying z 's parent to replace z by z 's child



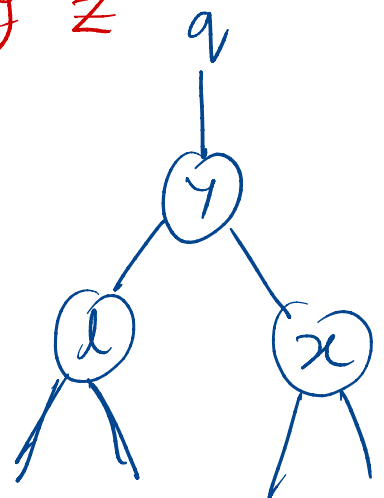
(III) If Z has 2 children, find Z 's successor Y — which must belong to Z 's right subtree — and move Y to take Z 's position in the tree

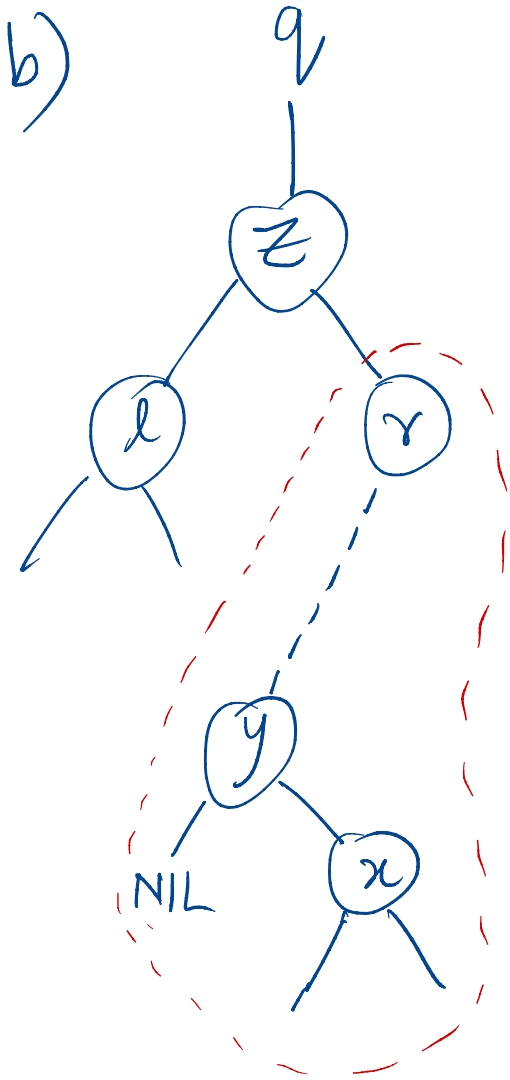
The rest of the Z 's original right subtree becomes Y 's new right subtree and Z 's left subtree becomes Y 's new left subtree.

a)

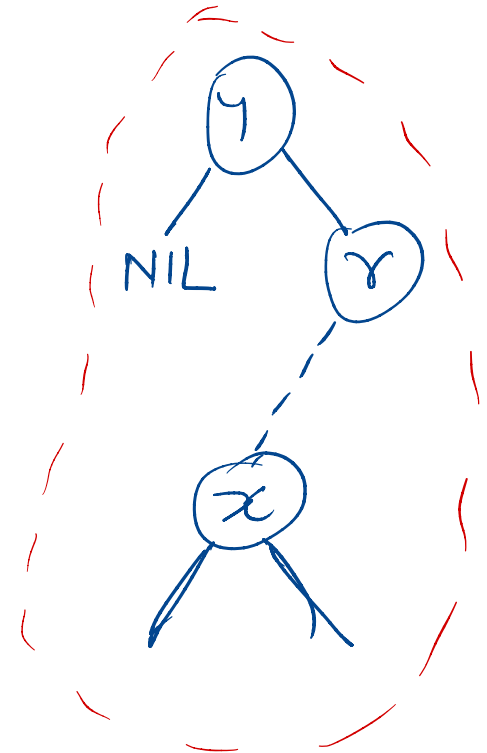
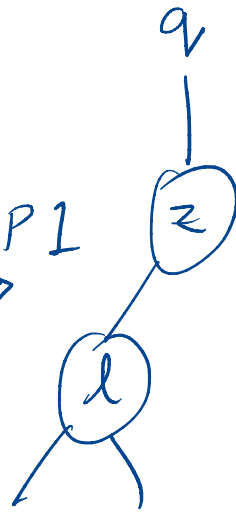


Case, when the successor of Z is the right child of Z



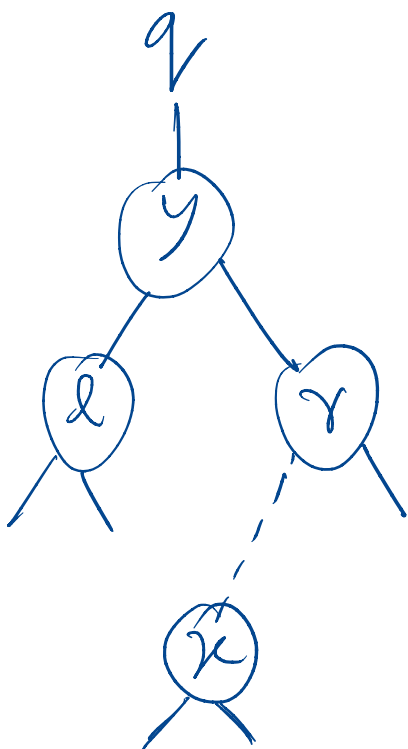


STEP 1
→



⇓

STEP 2
↓



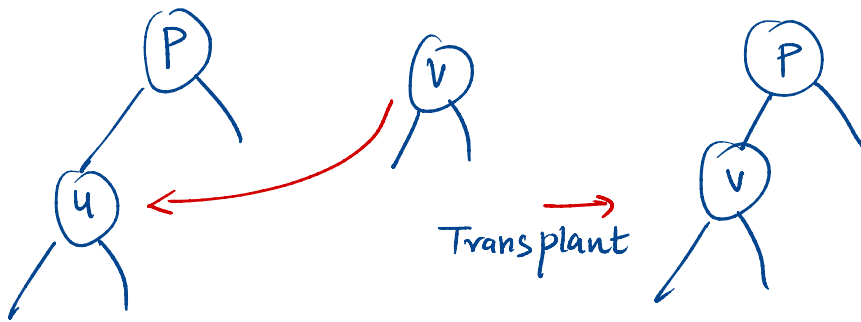
— remove y from the tree
(y has only 1 child x , so
replace y with x in the tree)

— place y as the root with x being
the right child of y
(this transformation keeps y
the smallest element)

Transplant Operation

↳ replaces one subtree as a child of its parent with another subtree

↳ Define transplant that replaces subtree rooted at node u with a subtree rooted at node v . set parent pointers appropriately.



Transplant (T, u, v)

if $u.p == \text{NIL}$
 $T.\text{root} = v$

else if $u == u.p.\text{left}$
 $u.p.\text{left} = v$

else
 $u.p.\text{right} = v$

if $v \neq \text{NIL}$

$v.p = u.p$ // set the parent

Define node deletion using the transplant operation

Tree-Delete (T, z)

handle cases:
- No child
- only 1 child

if $z.\text{left} == \text{NIL}$ → replace z by its right child
 Transplant (T, z, z.right)

else if $z.\text{right} == \text{NIL}$ → replace z by its left child.
 Transplant (T, z, z.left)

else
 → successor of z
 $y = \text{Tree-Minimum} (z.\text{right})$
 if $y \neq z.\text{right}$
 Transplant (T, y, y.right)
 $y.\text{right} = z.\text{right}$
 $y.\text{right}.p = y$
 → replace z by its successor
 Transplant (T, z, y)
 $y.\text{left} = z.\text{left}$
 $y.\text{left}.p = y$

handle case of both children
 left has to be NIL
 make y the right child of z

→ $O(h)$ complexity